Machine Learning Techniques

Unit 4

Ankita Singh, Dept. of IT, AKGEC

Syllabus

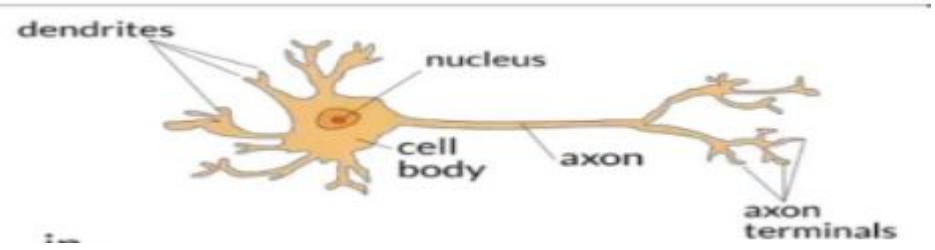
ARTIFICIAL NEURAL NETWORKS – Perceptron, Multilayer perceptron, Gradient descent and the Delta rule, Multilayer networks, Derivation of Backpropagation Algorithm,Generalization, Unsupervised Learning – SOM Algorithm and its variant;

DEEP LEARNING - Introduction,concept of convolutional neural network , Types of layers –(Convolutional Layers , Activation function , pooling , fully connected) , Concept of Convolution (1D and 2D) layers, Training of network, Case study of CNN for eg on Diabetic Retinopathy, Building a smart speaker, Self-driving car etc.

Introduction

- A neural network is a machine learning program, or model, that makes decisions in a manner similar to the human brain by using processes that mimic the way biological neurons work together to identify phenomena, weigh options and arrive at conclusions.
- Artificial Neural Networks work collectively and concurrently, imitating biological neural networks. They are used in many different fields, like regression problems, pattern recognition, recommendation systems, and weather forecasting.
- Tasks in speech recognition or image recognition can take minutes versus hours when compared to the manual identification by human experts. One of the best-known examples of a neural network is Google's search algorithm.

Structure of a Biological Neuron:



- **Dendrite (Input):**

  Receives signals from other neurons.

- **Cell Body (Summing Unit):**

  Sums up all the incoming input signals.

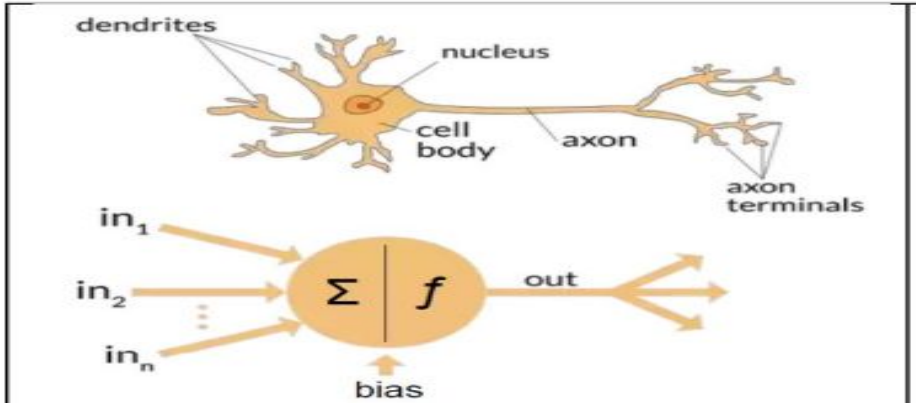
  Contains the threshold unit.

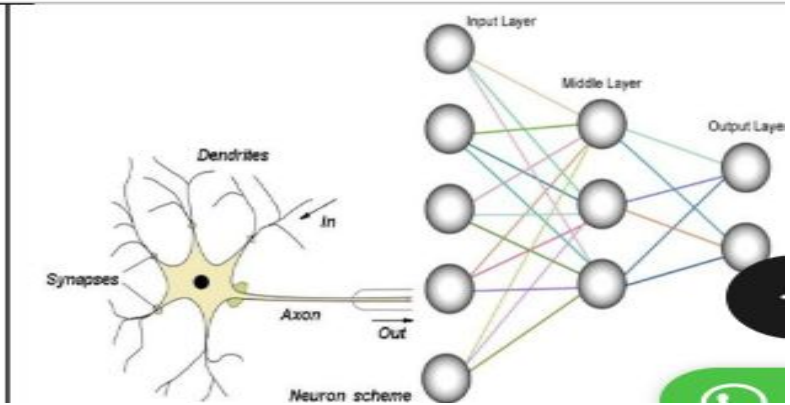
- **Axon (Output):**

  When the sum reaches a threshold level, the neuron generates an output signal which travels down to other neurons.

Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks. These neurons are known as nodes.

Biological Neuron	Artificial Neuron
Cell Nucleus (Soma)	Node
Dendrites	Input
Synapse	Weights or interconnections
Axon	Output





Artificial Neuron

An artificial neuron is a mathematical function based on the model of biological neurons.

The model of an artificial neuron has five components:

• **Input Values/Input Layer**: Input values are passed to a neuron using this layer. Similar to dendrites in biological neurons.

• **Synapses/Weighted Connection Links**: Each connection link is characterized by a weight or strength (wi). Each input (xi) is multiplied by its weight to get the synaptic strength.

• **Summing Function:**The summing function takes the sum of all weighted connection links: Σwi xi. A bias value (b) is also added to this weighted sum.

• **Activation Function:** The activation function decides whether or not a neuron is fired based on the output value produced.

• **Output/Output Layer**: The output layer gives the final output of a neuron which can then be passed to other neurons in the network.

Perceptron

- The first neural network model ‘Perceptron’, was designed in 1958
- Perceptron is a type of neural network that performs **binary classification,** that maps input features to an output decision, usually classifying data into one of two categories, such as 0 or 1.
- Perceptron consists of a **single layer of input nodes** that are fully connected to a layer of output nodes. It is particularly good at learning **linearly separable patterns**.

Types of Perceptron

1. **Single-Layer Perceptron** is a type of perceptron is limited to learning linearly separable patterns. It is effective for tasks where the data can be divided into distinct categories through a straight line. While powerful in its simplicity, it struggles with more complex problems where the relationship between inputs and outputs is non-linear.
2. **Multi-Layer Perceptron** possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.

Single Layer Perceptron

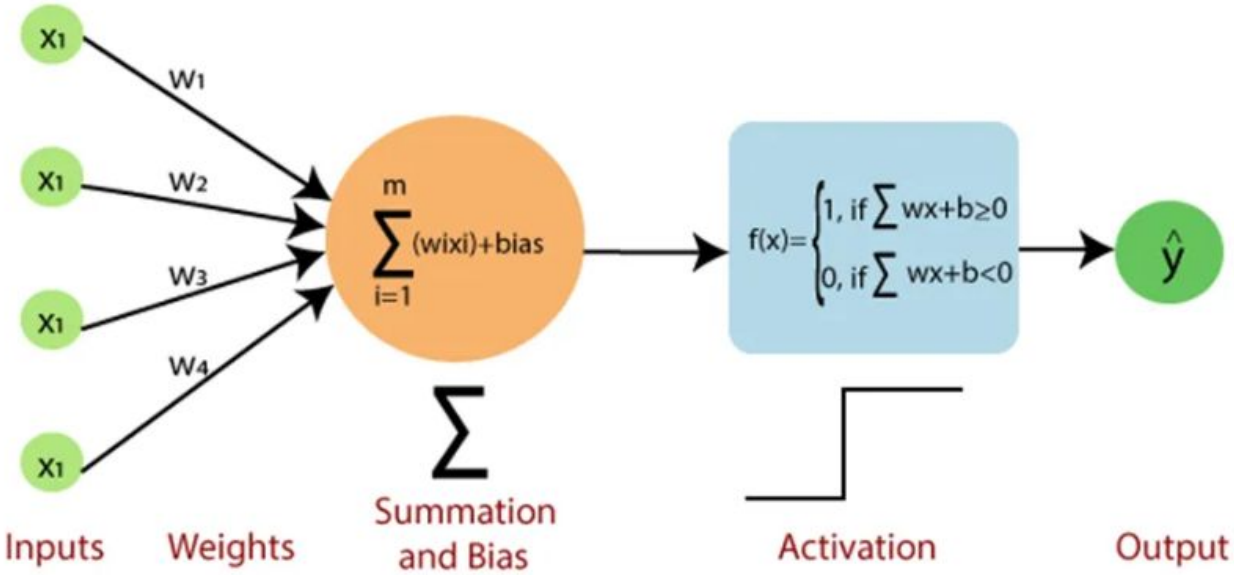
- A **perceptron** is the earliest form of a neural network unit, introduced by **Frank Rosenblatt** in 1958.
- It is a **binary classifier** that makes predictions based on a linear combination of input features.
- The perceptron algorithm was one of the first algorithms used to implement a simple neural network.
- **Perceptrons are supervised learning algorithms and is a type of an ANN**

Components of a Perceptron:

- **Inputs**: The perceptron takes several inputs (x1,x2,...,xn) each representing a feature of data.
- **Weights**: Each input is associated with a weight (w1,w2,...,wn) that represents its importance in the decision making process.
- **Bias**: A bias term (b) is added to shift the decision boundary.
- **Activation Function**: A function that processes the sum of weighted inputs and bias to produce an output.The perceptron uses a **step function** (a simple thresholding function) to determine whether the weighted sum of inputs plus the bias is above or below a certain threshold to give the result.

The mathematical representation is:

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{n} w_i x_i + b > 0 \\ 0 & \text{if } \sum_{i=1}^{n} w_i x_i + b \leq 0 \end{cases}$$


X1
X1
X1
X1
W1
W2
W3
W4
m
∑ (wixi)+bias
i=1
f(x)= 1, if ∑ wx+b≥0
0, if ∑ wx+b<0
ŷ
Inputs
Weights
Summation and Bias
Activation
Output

Perceptron Learning Algorithm

Set initial weights $w_1, w_2, \ldots, w_n$ and bias b to a random value in the range $[-0.5, 0.5]$.

For each Epoch:

1. Compute the weighted sum by multiplying the inputs with the weights and add the products.
2. Apply the activation function on the weighted sum:

$$Y = \text{Step}\left(\sum w_i x_i + b\right)$$

3. If the sum is above the threshold value, output the value as positive else output the value as negative.
4. Calculate the error by subtracting the estimated output $Y_{estimated}$ from the desired output $Y_{desired}$:

$$error\ e(t) = Y_{desired} - Y_{estimated}$$

5. Update the weights if there is an error:

$$\Delta w_i = \alpha \times e(t) \times x_i$$

$$w_i = w_i + \Delta w_i$$

where x_i is the input value, $e(t)$ is the error at step t, α is the learning rate, and Δw_i is the difference in weight that has to be added to w_i.

AND function using Perceptron Rule Solved Example

$$y_{in} = b + x_1 w_1 + x_2 w_2$$

$$w_i \leftarrow w_i + \alpha\,(t - y)\,x_i, \qquad b \leftarrow b + \alpha\,(t - y). \qquad y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

Learning rate (α) = 1 Initially b=0, w_1=0, w_2=0

x_1	x_2	target	y_{in}	y	Error (t-y)	w_1	w_2	b
1	1	1	0	0	1	1	1	1
1	-1	-1	1	1	-2	-1	3	-1
-1	1	-1	1	1	-2	1	1	-3
-1	-1	-1	-5	-1	0	1	1	-3

End of Epoch 1: $w_1 = 1,\ w_2 = 1,\ b = -3.$

AND function using Perceptron Rule Solved Example

$$y_{in} = b + x_1 w_1 + x_2 w_2$$

$$w_i \leftarrow w_i + \alpha\,(t - y)\,x_i, \qquad b \leftarrow b + \alpha\,(t - y). \qquad y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

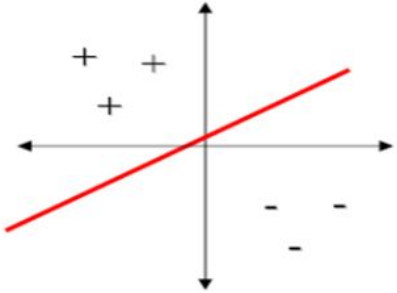
Learning rate (α) = 1

x_1	x_2	target	y_{in}	y	error(t -y)	w_1	w_2	b
1	1	1	-1	-1	2	3	3	-1
1	-1	-1	-1	-1	0	3	3	-1
-1	1	-1	-1	-1	0	3	3	-1
-1	-1	-1	-7	-1	0	3	3	-1

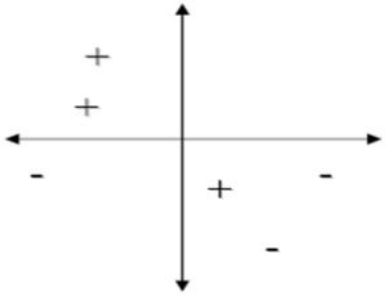
Final Weights after Convergence

$$\boxed{w_1 = 3, \quad w_2 = 3, \quad b = -1}$$

Gradient Descent and Delta Rule



Linearly separable



Non-linearly separable

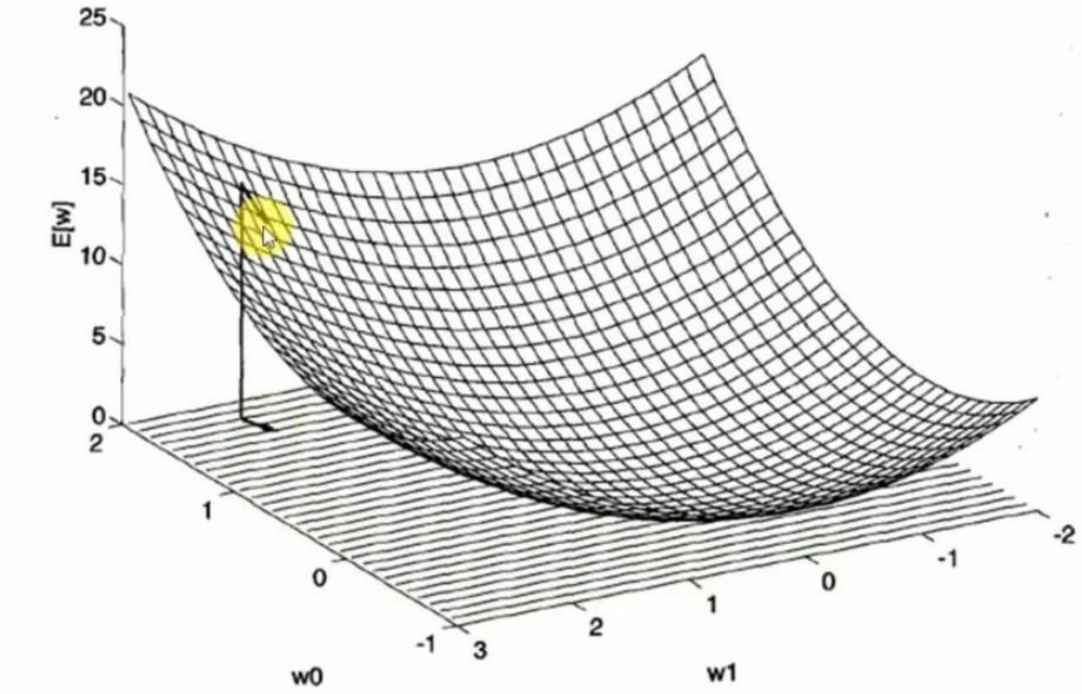
A set of data points are said to be linearly separable if the data can be divided into two classes using a straight line. If the data is not divided into two classes using a straight line, such data points are said to be called non-linearly separable data.

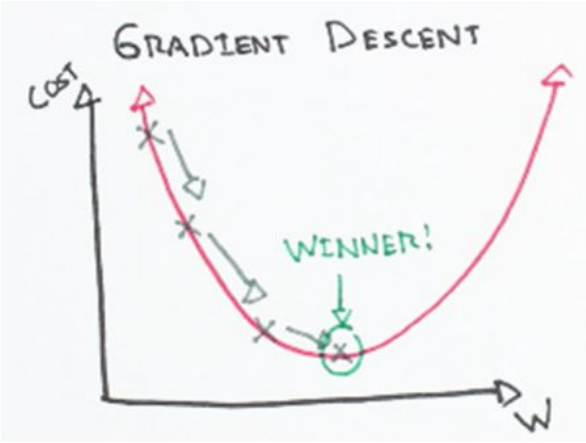
- Although the perceptron rule finds a successful weight vector when the training examples are linearly separable, it can fail to converge if the examples are not linearly separable.

- The perceptron rule works well for gates like **AND**, **OR**, and **NAND**, since their outputs are **linearly separable**, allowing the algorithm to find a clear decision boundary. However, it fails for gates such as **XOR** , where the outputs are **not linearly separable** — no single straight line can divide the classes correctly. Hence, the perceptron **converges** for linearly separable gates but **fails to converge** for non-linearly separable ones.

- A second training rule, called the **delta rule**, is designed to overcome this difficulty.
- If the training examples are not linearly separable, the delta rule converges toward a best-fit approximation to the target concept.
- The key idea behind the delta rule is to use **gradient descent to search the hypothesis space of possible weight vectors to find the weights that best fit the training examples.**
- This rule is important because gradient descent provides the basis for the BACKPROPAGATON algorithm, which can learn networks with many interconnected units.

Gradient Descent and the Delta Rule





Derivation of Delta Rule

- The delta training rule is best understood by considering the task of training an unthresholded perceptron; that is, a linear unit for which the output o is given by

$$o = w_0 + w_1 x_1 + \cdots + w_n x_n$$

$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$

- Thus, a linear unit corresponds to the first stage of a perceptron, without the threshold.
- In order to derive a weight learning rule for linear units, let us begin by specifying a measure for the training error of a hypothesis (weight vector), relative to the training examples.
- Although there are many ways to define this error, one common measure is $E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$

where D is the set of training examples, 'td' is the target output for training example 'd', and od is the output of the linear unit for training example 'd'.

How to calculate the direction of steepest descent along the error surface?

The direction of steepest can be found by computing the derivative of E with respect to each component of the vector w. This vector derivative is called the gradient of E with respect to w, written as,

$$\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots \frac{\partial E}{\partial w_n}\right]$$

The gradient specifies the direction of steepest increase of E, the training rule for gradient descent is

$$\vec{w} \leftarrow \vec{w} + \Delta\vec{w}$$

where

$$\Delta\vec{w} = -\eta\nabla E(\vec{w})$$

Here η is a positive constant called the learning rate, which determines the step size in the gradient descent search.

The negative sign is present because we want to move the weight vector in the direction that decreases E.

Here,

$$\Delta \vec{w} = -\eta \nabla E(\vec{w})$$

$$\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots \frac{\partial E}{\partial w_n}\right]$$

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$

$$\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_{d \in D} \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\
&= \frac{1}{2} \sum_{d \in D} 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\
&= \sum_{d \in D} (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \vec{w} \cdot \vec{x}_d) \\
\frac{\partial E}{\partial w_i} &= \sum_{d \in D} (t_d - o_d)(-x_{id})
\end{aligned}$$

Finally,

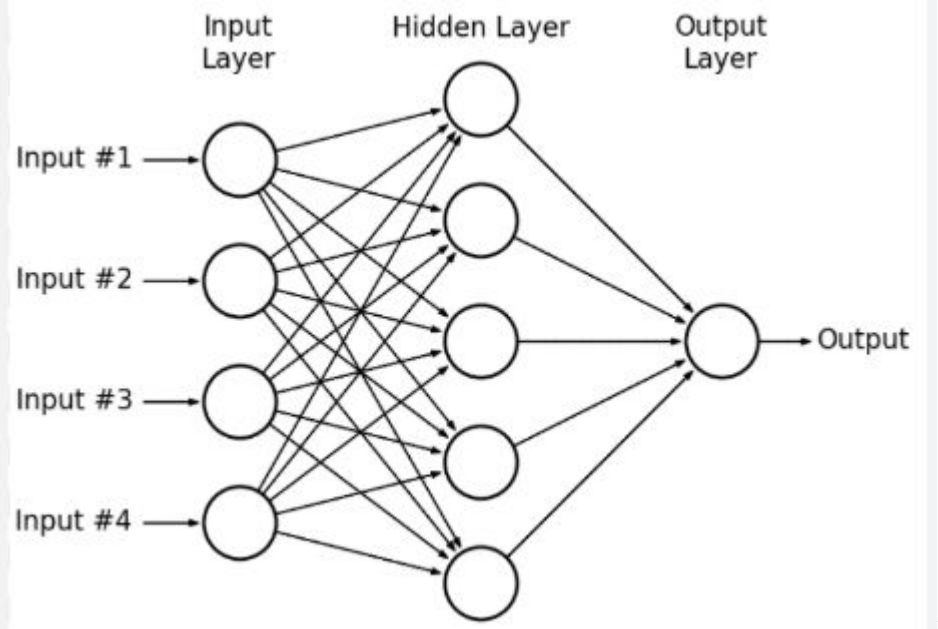
$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d)\ x_{id}$$

$$w_i \leftarrow w_i + \Delta w_i$$

Multi-layer Perceptron

A multi-layer perceptron is a type of Feed Forward Neural Network with multiple neurons arranged in layers.



The network has at least three layers:

- **Input Layer**: Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers**: MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer**: The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.

Every connection in the diagram is a representation of the **fully connected nature** of an MLP. This means that every node in one layer connects to every node in the next layer.

As the data moves through the network each layer transforms it until the final output is generated in the output layer

Working of Multi-Layer Perceptron

The key mechanisms such as forward propagation, loss function, backpropagation and optimization.

1. <u>Forward Propagation</u>

In forward propagation the data flows from the input layer to the output layer, passing through any hidden layers. Each neuron in the hidden layers processes the input as follows:

a. Weighted Sum: The neuron computes the weighted sum of the inputs: $z = \sum_i w_i x_i + b$

b. Activation Function: The weighted sum z is passed through an activation function to introduce non-linearity. Common activation functions include: **Sigmoid, ReLU(Rectified Linear Unit), Tanh(Hyperbolic Tangent)**

2. Loss Function

- Once the network generates an output the next step is to calculate the loss using a loss function. In supervised learning this compares the predicted output to the actual label.
- A loss/error is computed using functions like **MSE** (for regression) or **Cross-Entropy** (for classification).
- This loss indicates how far the prediction is from the expected answer.

3. Backpropagation

- The goal of training an MLP is to minimize the loss function by adjusting the network's weights and biases. This is achieved through backpropagation
- Error is propagated backward from output layer to input layer.
- Gradients (partial derivatives) are computed using **chain rule**.
- Weights and biases are updated using an optimization method like **Gradient Descent** to reduce loss.
- The process repeats across many epochs until the network learns the correct mapping

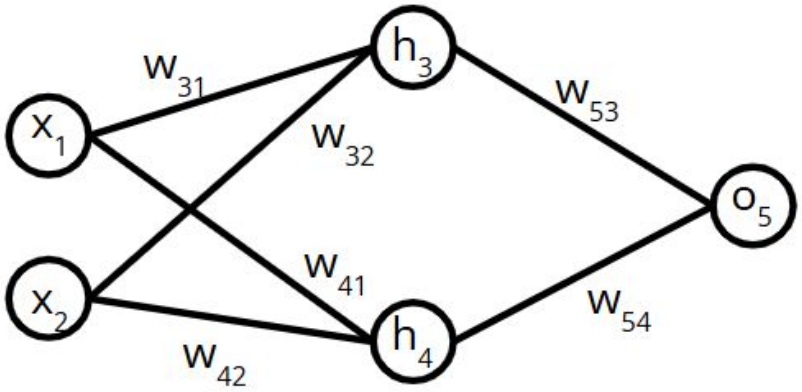
Example

x_1, x_2: input to the perceptron

t : expected output / target

η: learning rate

w : weights

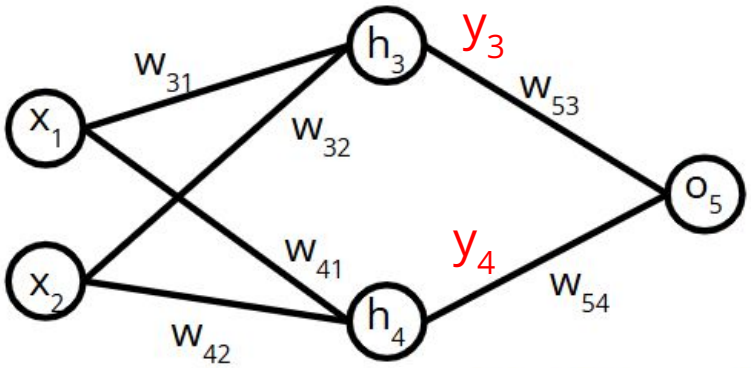


$$x_1 = 0.35,\ x_2 = 0.90,\ t = 0.5,\ \eta = 1$$

Initial weights:

$$w_{31} = 0.1,\ w_{32} = 0.8,\ w_{41} = 0.4,\ w_{42} = 0.6,\ w_{53} = 0.3,\ w_{54} = 0.9$$

Activation: $f(net) = \sigma(net) = \frac{1}{1+e^{-net}}$



$$x_1 = 0.35,\ x_2 = 0.90$$

$$w_{31} = 0.1,\ w_{32} = 0.8,\ w_{41} = 0.4,\ w_{42} = 0.6,$$

Activation: $f(net) = \sigma(net) = \frac{1}{1+e^{-net}}$

- **Step 1: Forward pass**

Hidden unit 3

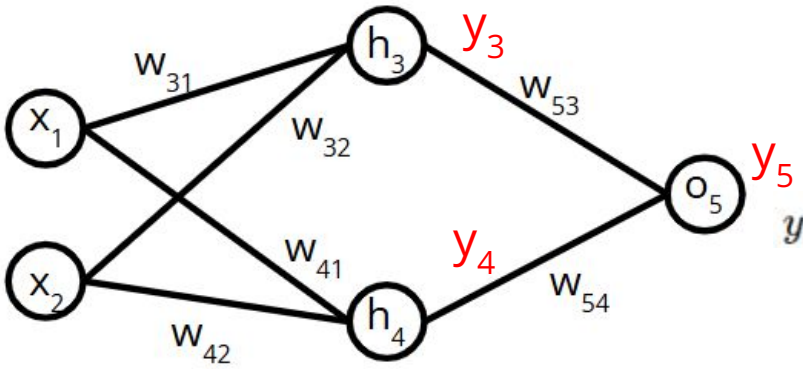
$$net_3 = w_{31}x_1 + w_{32}x_2 = (0.1)(0.35) + (0.8)(0.9) = 0.755$$

$$y_3 = \sigma(net_3) = 0.6803$$

Hidden unit 4

$$net_4 = w_{41}x_1 + w_{42}x_2 = (0.4)(0.35) + (0.6)(0.9) = 0.68$$

$$y_4 = \sigma(net_4) = 0.6637$$



$$w_{53} = 0.3, \; w_{54} = 0.9$$

$$y_3 = \sigma(\text{net}_3) = 0.6803. \qquad y_4 = \sigma(\text{net}_4) = 0.6637.$$

$$t = 0.5$$

Output unit 5

$$net_5 = w_{53}y_3 + w_{54}y_4 = (0.3)(0.6803) + (0.9)(0.6637) = 0.8014$$

$$y_5 = \sigma(net_5) = 0.6903$$

Error:

$$E = \frac{1}{2}(t - y_5)^2 = 0.0181$$

Minimize this error, and propagate it backwards to update the weights

Backpropagation

- Backpropagation, short for Backward Propagation of Errors, is a key algorithm used to train neural networks by minimizing the difference between predicted and actual outputs.
- It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases.

◆ **Chain Rule (Basic Idea)**

If a variable y depends on u, and u depends on x,

then y depends indirectly on x.

Mathematically:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

- Combined with optimization techniques like gradient descent, backpropagation enables the model to reduce loss across epochs and effectively learn complex patterns from data.

for each training example d descending the gradient of the error E_d with respect to this single example. In other words, for each training example d every weight w_{ji} is updated by adding to it Δw_{ji}

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} \tag{4.21}$$

where E_d is the error on training example d, summed over all output units in the network

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in outputs} (t_k - o_k)^2$$

Here *outputs* is the set of output units in the network, t_k is the target value of unit k for training example d, and o_k is the output of unit k given training example d.

chain rule to write

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}}$$

$$= \frac{\partial E_d}{\partial net_j} x_{ji} \tag{4.22}$$

Given Equation (4.22), our remaining task is to derive a convenient expression for $\frac{\partial E_d}{\partial net_j}$. We consider two cases in turn: the case where unit j is an output unit for the network, and the case where j is an internal unit.

Case 1: Training Rule for Output Unit Weights. Just as w_{ji} can influence the rest of the network only through net_j, net_j can influence the network only through o_j. Therefore, we can invoke the chain rule again to write

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j}\frac{\partial o_j}{\partial net_j} \tag{4.23}$$

To begin, consider just the first term in Equation (4.23)

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j}\frac{1}{2}\sum_{k \in outputs}(t_k - o_k)^2$$

The derivatives $\frac{\partial}{\partial o_j}(t_k - o_k)^2$ will be zero for all output units k except when $k = j$. We therefore drop the summation over output units and simply set $k = j$.

$$\begin{aligned}\frac{\partial E_d}{\partial o_j} &= \frac{\partial}{\partial o_j}\frac{1}{2}(t_j - o_j)^2 \\ &= \frac{1}{2}2(t_j - o_j)\frac{\partial(t_j - o_j)}{\partial o_j} \\ &= -(t_j - o_j) \end{aligned} \tag{4.24}$$

Next consider the second term in Equation (4.23). Since $o_j = \sigma(net_j)$, the derivative $\frac{\partial o_j}{\partial net_j}$ is just the derivative of the sigmoid function, which we have already noted is equal to $\sigma(net_j)(1 - \sigma(net_j))$. Therefore,

$$\begin{aligned} \frac{\partial o_j}{\partial net_j} &= \frac{\partial \sigma(net_j)}{\partial net_j} \\ &= o_j(1 - o_j) \end{aligned} \tag{4.25}$$

Substituting expressions (4.24) and (4.25) into (4.23), we obtain

$$\frac{\partial E_d}{\partial net_j} = -(t_j - o_j)\, o_j(1 - o_j) \tag{4.26}$$

and combining this with Equations (4.21) and (4.22), we have the stochastic gradient descent rule for output units

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta\ (t_j - o_j)\ o_j(1 - o_j)x_{ji} \tag{4.27}$$

Note this training rule is exactly the weight update rule implemented by Equations (T4.3) and (T4.5) in the algorithm of Table 4.2. Furthermore, we can see now that δ_k in Equation (T4.3) is equal to the quantity $-\frac{\partial E_d}{\partial net_k}$. In the remainder of this section we will use δ_i to denote the quantity $-\frac{\partial E_d}{\partial net_i}$ for an arbitrary unit i.

Training rule for Output unit weights $\delta_k = (t_j - o_j)\ o_j(1 - o_j)$.

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta\, \delta_k\, x_{ji}$$

Case 2: Training Rule for Hidden Unit Weights. In the case where j is an internal, or hidden unit in the network, the derivation of the training rule for w_{ji} must take into account the indirect ways in which w_{ji} can influence the network outputs and hence E_d. For this reason, we will find it useful to refer to the set of all units immediately downstream of unit j in the network (i.e., all units whose direct inputs include the output of unit j). We denote this set of units by $Downstream(j)$. Notice that net_j can influence the network outputs (and therefore E_d) only through the units in $Downstream(j)$. Therefore, we can write

$$\begin{aligned}
\frac{\partial E_d}{\partial net_j} &= \sum_{k \in Downstream(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j} \\
&= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial net_j} \\
&= \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial o_j} \frac{\partial o_j}{\partial net_j} \\
&= \sum_{k \in Downstream(j)} -\delta_k \, w_{kj} \frac{\partial o_j}{\partial net_j} \\
&= \sum_{k \in Downstream(j)} -\delta_k \, w_{kj} \, o_j(1 - o_j)
\end{aligned} \tag{4.28}$$

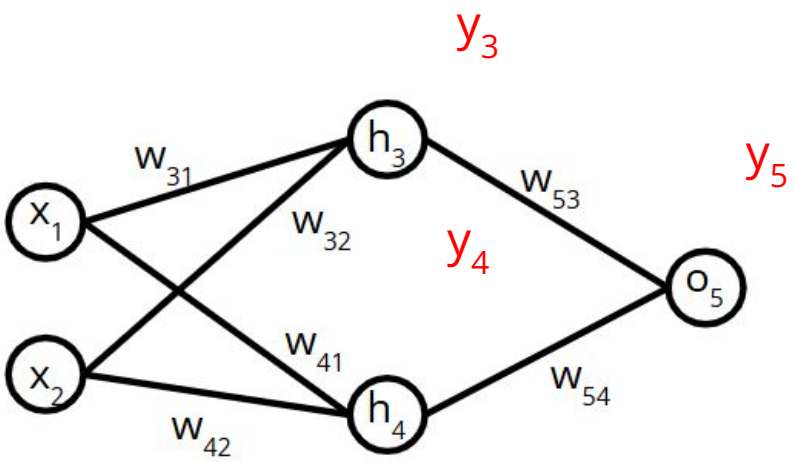
Rearranging terms and using δ_j to denote $-\frac{\partial E_d}{\partial net_j}$, we have

$$\delta_j = o_j(1 - o_j) \sum_{k \in Downstream(j)} \delta_k \; w_{kj}$$

and

$$\Delta w_{ji} = \eta \; \delta_j \; x_{ji}$$

Example Continued....



$$t = 0.5$$

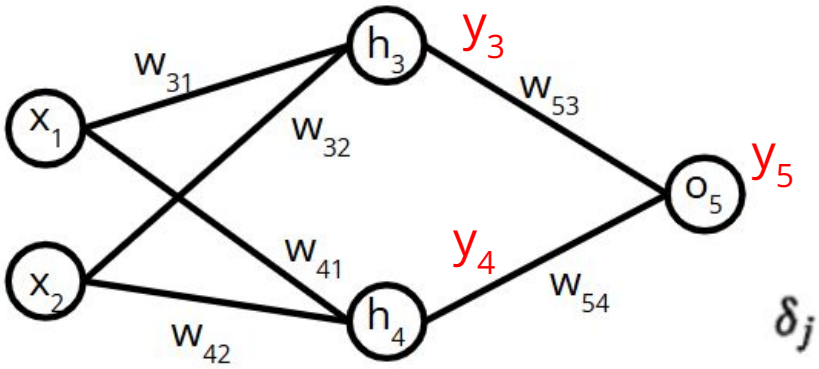
$$y_5 = \sigma(net_5) = 0.6903$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta\ (t_j - o_j)\ o_j(1 - o_j)x_{ji} \qquad (4.27)$$

- **Step 2: Output delta (unit 5)**

Mitchell: $\delta_5 = (t - y_5)f'(net_5) = (t - y_5)y_5(1 - y_5)$

$$\delta_5 = (0.5 - 0.6903)(0.6903)(1 - 0.6903) = -0.0407$$



$$t = 0.5$$

$$y_5 = \sigma(net_5) = 0.6903$$

$$\delta_j = o_j(1 - o_j) \sum_{k \in Downstream(j)} \delta_k \; w_{kj}$$

- **Step 3: Hidden deltas**

$$\delta_j = f'(net_j) \sum_k w_{kj} \delta_k$$

For hidden unit 3:

$$\delta_3 = y_3(1 - y_3)(w_{53}\delta_5) = 0.6803(1 - 0.6803)(0.3)(-0.0407) = -0.0027$$

For hidden unit 4:

$$\delta_4 = y_4(1 - y_4)(w_{54}\delta_5) = 0.6637(1 - 0.6637)(0.9)(-0.0407) = -0.0082$$

Step 4: Weight updates

Rule:

$$\Delta w_{ji} = \eta \delta_j x_i$$

for input → hidden, and

$$\Delta w_{kj} = \eta \delta_k y_j$$

for hidden → output.

Hidden → Output

$$\Delta w_{53} = \eta \delta_5 y_3 = (-0.0407)(0.6803) = -0.0277$$

$$\Delta w_{54} = (-0.0407)(0.6637) = -0.0270$$

$$w_{53}^{new} = 0.3 - 0.0277 = 0.2723$$

$$w_{54}^{new} = 0.9 - 0.0270 = 0.8730$$

Input → Hidden

For hidden 3:

$$\Delta w_{31} = \eta \delta_3 x_1 = (-0.0027)(0.35) = -0.0009$$

$$\Delta w_{32} = (-0.0027)(0.9) = -0.0024$$

$$w_{31}^{new} = 0.1 - 0.0009 = 0.0991$$

$$w_{32}^{new} = 0.8 - 0.0024 = 0.7976$$

For hidden 4:

$$\Delta w_{41} = (-0.0082)(0.35) = -0.0029$$

$$\Delta w_{42} = (-0.0082)(0.9) = -0.0074$$

$$w_{41}^{new} = 0.4 - 0.0029 = 0.3971$$

$$w_{42}^{new} = 0.6 - 0.0074 = 0.5926$$

Generalization

- Generalization refers to the ability of a trained neural network to perform well on unseen data that was not part of the training set.
- When training a neural network, there's going to be some data that the neural network trains on, and there's going to be some data reserved for checking the performance of the neural network. If the neural network performs well on the data which it has not trained on, we can say it has generalized well on the given data
- Instead of memorizing the training examples, the network learns the underlying patterns and relationships in the data.
- Good generalization ensures that the model predicts accurately in real-world scenarios.

- Generalization is important because the true test of a model's effectiveness is not how well it performs on the training data, but rather how well it generalizes to new and unseen data.
- If a model fails to generalize, it may exhibit high accuracy on the training set but will likely perform poorly on real-world examples.
- This limitation renders the model impractical and unreliable in practical applications.
- To achieve successful generalization, machine learning practitioners must address challenges like overfitting and underfitting.

Factors to achieve Generalization

- **Balanced model complexity**: A model that is too simple underfits and fails to learn patterns, while an overly complex model memorizes data and overfits.
- **Regularization techniques**: Methods like L2 regularization, dropout, and early stopping help prevent overfitting.
- **Sufficient and diverse training data**: More representative data helps the network capture true patterns.
- **Validation**: Testing performance on separate validation data ensures the model learns general behavior, not noise.

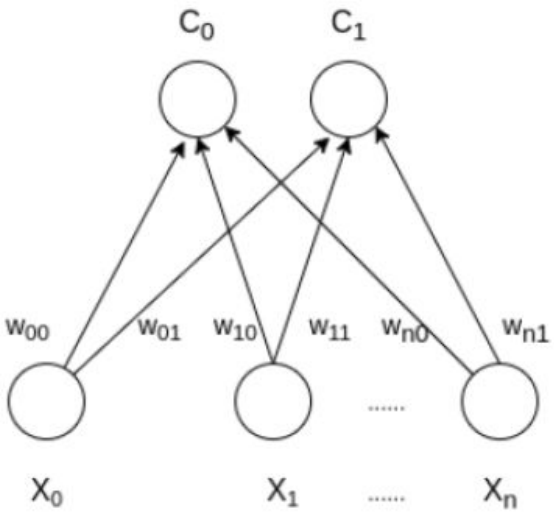
Self Organizing Map (SOM) or Kohonen Map

- A Self Organizing Map (SOM) or Kohonen Map is an **unsupervised neural network algorithm** based on biological neural models from the 1970s.
- It uses a competitive learning approach and is primarily designed for **clustering and dimensionality reduction**.
- SOM effectively maps high-dimensional data to a lower-dimensional grid enabling easier interpretation and visualization of complex datasets.

Ankita Singh, Dept. of IT, AKGEC

It consists of two primary layers:

- **Input Layer:** Represents the features of the data.
- **Output Layer**: Arranged as a 2D grid of neurons with each neuron representing a cluster in the data.



Working of Self Organizing Maps (SOM)

1. Initialization: The weights of the output neurons are randomly initialized. These weights represent the features of each neuron and will be adjusted during training.

2. Competition: For each input vector, SOM computes the Euclidean distance between the input and the weight vectors of all neurons. The neuron with the smallest distance is the winning neuron.

Formula : $D(j) = \sum_{i=1}^{n} (w_{ij} - x_i)^2$

where

- $D(j)$ is the distance for neuron
- j and n is the number of features.

Ankita Singh, Dept. of IT, AKGEC

3. Weight Update: The winning neuron's weights are updated to move closer to the input vector. The weights of neighboring neurons are also adjusted but with smaller changes.

Formula: $w_{ij}^{(new)} = w_{ij}^{(old)} + \alpha \cdot (x_i - w_{ij}^{(old)})$

where

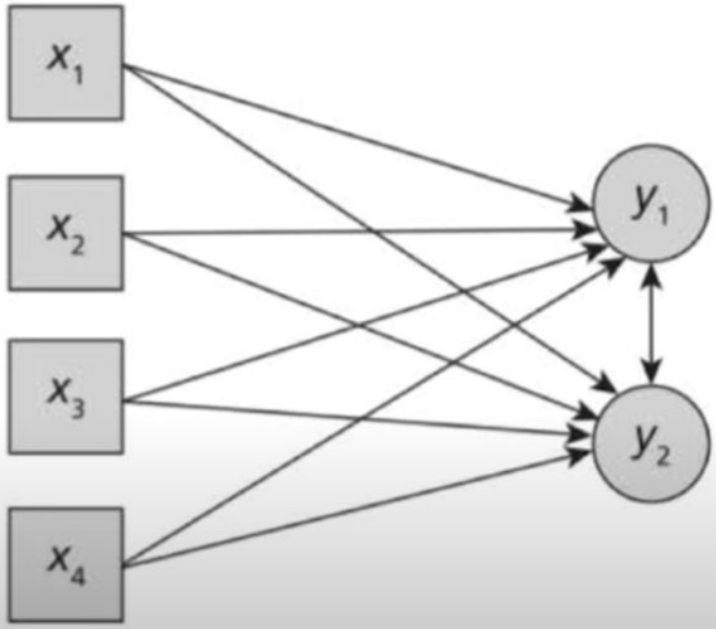
- α is the learning rate
- x_i is the input feature.

4. Learning Rate Decay: The learning rate α decreases over time allowing the map to converge to stable values.

Formula: $\alpha(t+1) = 0.5 \cdot \alpha(t)$

5. Stopping Condition: The training stops when the maximum number of epochs is reached or when the weights converge.

- Consider the network shown in Figure which considers four training samples each vector of length 4 and two output units.
- Train the SOFM network by determining the class memberships of the input data
- Training Samples:
- X1: (1, 0, 1, 0) X2: (1, 0, 0, 0)
- X3: (1, 1, 1, 1) X4: (0, 1, 1, 0)



Example

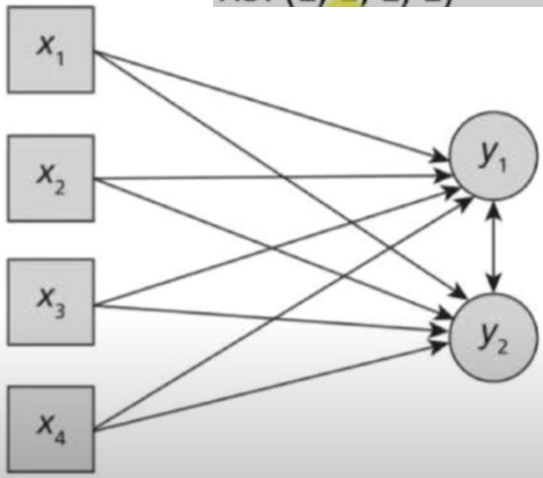
Consider the network shown which considers 4 training samples each vector of length 4 and 2 output units. Train SOFM network by determining the class membership of input data

Training Samples:

X1: (1, 0, 1, 0) X2: (1, 0, 0, 0)

X3: (1, 1, 1, 1) X4: (0, 1, 1, 0)

- Output Units: Unit 1, Unit 2
- Learning rate η(t) = 0.6
- Initial Weight matrix
- $\begin{bmatrix} Unit\ 1 \\ Unit\ 2 \end{bmatrix} = \begin{bmatrix} 0.3 & 0.5 & 0.7 & 0.2 \\ 0.6 & 0.5 & 0.4 & 0.2 \end{bmatrix}$



Iteration 1:

Training Sample x_1: (1, 0, 1, 0)

Weight matrix:

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.3 & 0.5 & 0.7 & 0.2 \\ 0.6 & 0.7 & 0.4 & 0.3 \end{bmatrix}$$

Compute Euclidean distance between x_1: (1, 0, 1, 0) and Unit 1 weights.

$$d^2 = (0.3 - 1)^2 + (0.5 - 0)^2 + (0.7 - 1)^2 + (0.2 - 0)^2 = 0.87$$

Compute Euclidean distance between x_1: (1, 0, 1, 0) and Unit 2 weights.

$$d^2 = (0.6 - 1)^2 + (0.7 - 0)^2 + (0.4 - 1)^2 + (0.3 - 0)^2 = 1.1$$

Unit 1 wins

Formula: $w_{ij}^{(new)} = w_{ij}^{(old)} + \alpha \cdot (x_i - w_{ij}^{(old)})$

Update the weights of the winning unit.

$$
\begin{aligned}
\text{New Unit 1 weights} &= [0.3\ \ 0.5\ \ 0.7\ \ 0.2] + 0.6\,([1\ \ 0\ \ 1\ \ 0] - [0.3\ \ 0.5\ \ 0.7\ \ 0.2]) \\
&= [0.3\ \ 0.5\ \ 0.7\ \ 0.2] + 0.6\,[0.7\ \ -0.5\ \ 0.3\ \ -0.2] \\
&= [0.3\ \ 0.5\ \ 0.7\ \ 0.2] + [0.42\ \ -0.30\ \ 0.18\ \ -0.12] \\
&= [0.72\ \ 0.2\ \ 0.88\ \ 0.08]
\end{aligned}
$$

$$
\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.72 & 0.2 & 0.88 & 0.08 \\ 0.6 & 0.7 & 0.4 & 0.3 \end{bmatrix}
$$

Iteration 2:

Training Sample x_2: (1, 0, 0, 0)

Weight matrix:

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.72 & 0.2 & 0.88 & 0.08 \\ 0.6 & 0.7 & 0.4 & 0.3 \end{bmatrix}$$

Compute Euclidean distance between x_2: (1, 0, 0, 0) and Unit 1 weights.

$$d^2 = (0.72 - 1)^2 + (0.2 - 0)^2 + (0.88 - 0)^2 + (0.08 - 0)^2 = 0.74$$

Compute Euclidean distance between x_2: (1, 0, 0, 0) and Unit 2 weights.

$$d^2 = (0.6 - 1)^2 + (0.7 - 0)^2 + (0.4 - 0)^2 + (0.3 - 0)^2 = 0.9$$

Unit 1 wins

Formula: $w_{ij}^{(new)} = w_{ij}^{(old)} + \alpha \cdot (x_i - w_{ij}^{(old)})$

Update the weights of the winning unit:

$$
\begin{aligned}
\text{New Unit 1 weights} &= [0.72 \;\; 0.2 \;\; 0.88 \;\; 0.08] + 0.6\,([1 \;\; 0 \;\; 0 \;\; 0] - [0.72 \;\; 0.2 \;\; 0.88 \;\; 0.08]) \\
&= [0.72 \;\; 0.2 \;\; 0.88 \;\; 0.08] + 0.6\,[0.28 \;\; -0.2 \;\; -0.88 \;\; -0.08] \\
&= [0.72 \;\; 0.2 \;\; 0.88 \;\; 0.08] + [0.17 \;\; -0.12 \;\; -0.53 \;\; -0.05] \\
&= [0.89 \;\; 0.08 \;\; 0.35 \;\; 0.03]
\end{aligned}
$$

$$
\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.89 & 0.08 & 0.35 & 0.03 \\ 0.6 & 0.7 & 0.4 & 0.3 \end{bmatrix}
$$

Iteration 3:

Training Sample x_3: (1, 1, 1, 1)

Weight matrix:

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.89 & 0.08 & 0.35 & 0.03 \\ 0.6 & 0.7 & 0.4 & 0.3 \end{bmatrix}$$

Compute Euclidean distance between x_3: (1, 1, 1, 1) and Unit 1 weights.

$$d^2 = (0.89 - 1)^2 + (0.08 - 1)^2 + (0.35 - 1)^2 + (0.03 - 1)^2$$
$$= 2.2$$

Compute Euclidean distance between x_3: (1, 1, 1, 1) and Unit 2 weights.

$$d^2 = (0.6 - 1)^2 + (0.7 - 1)^2 + (0.4 - 1)^2 + (0.3 - 1)^2$$
$$= 1.1$$

Unit 2 wins

Formula: $w_{ij}^{(new)} = w_{ij}^{(old)} + \alpha \cdot (x_i - w_{ij}^{(old)})$

Update the weights of the winning unit:

$$\begin{aligned}\text{New Unit 2 weights} &= [0.6\ \ 0.7\ \ 0.4\ \ 0.3] + 0.6\,([1\ \ 1\ \ 1\ \ 1] - [0.6\ \ 0.7\ \ 0.4\ \ 0.3]) \\ &= [0.6\ \ 0.7\ \ 0.4\ \ 0.3] + 0.6\,[0.4\ \ 0.3\ \ 0.6\ \ 0.7] \\ &= [0.6\ \ 0.7\ \ 0.4\ \ 0.3] + [0.24\ \ 0.18\ \ 0.36\ \ 0.42] = [0.84\ \ 0.88\ \ 0.76\ \ 0.72]\end{aligned}$$

$$\begin{bmatrix}\text{Unit 1}\\ \text{Unit 2}\end{bmatrix} : \begin{bmatrix}0.89 & 0.08 & 0.35 & 0.03\\ 0.84 & 0.88 & 0.76 & 0.72\end{bmatrix}$$

Iteration 4:

Training Sample x_4: (0, 1, 1, 0)

Weight matrix:

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.89 & 0.08 & 0.35 & 0.03 \\ 0.84 & 0.88 & 0.76 & 0.72 \end{bmatrix}$$

Compute Euclidean distance between x_4: (0, 1, 1, 0) and Unit 1 weights.

$$d^2 = (0.89 - 0)^2 + (0.08 - 1)^2 + (0.35 - 1)^2 + (0.03 - 0)^2$$

$$= 2.06$$

Compute Euclidean distance between x_1: (0, 1, 1, 0) and Unit 2 weights.

$$d^2 = (0.84 - 0)^2 + (0.88 - 1)^2 + (0.76 - 1)^2 + (0.72 - 0)^2$$

$$= 1.3$$

Formula: $w_{ij}^{(new)} = w_{ij}^{(old)} + \alpha \cdot (x_i - w_{ij}^{(old)})$

Unit 2 wins

Update the weights of the winning unit:

$$\begin{aligned}
\text{New Unit 2 weights} &= [0.84\ \ 0.88\ \ 0.76\ \ 0.72] + 0.6\,([0\ \ 1\ \ 1\ \ 0] - [0.84\ \ 0.88\ \ 0.76\ \ 0.72]) \\
&= [0.84\ \ 0.88\ \ 0.76\ \ 0.72] + 0.6\,[-0.84\ \ 0.12\ \ 0.24\ \ -0.72] \\
&= [0.84\ \ 0.88\ \ 0.76\ \ 0.72] + [-0.5\ \ 0.07\ \ 0.14\ \ -0.43] = [0.34\ \ 0.95\ \ 0.9\ \ 0.29]
\end{aligned}$$

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.89 & 0.08 & 0.35 & 0.03 \\ 0.34 & 0.95 & 0.9 & 0.29 \end{bmatrix}$$

Best mapping units for each of the sample taken are:

x_1: (1, 0, 1, 0) → Unit 1

x_2: (1, 0, 0, 0) → Unit 1

x_3: (1, 1, 1, 1) → Unit 2

x_4: (0, 1, 1, 0) → Unit 2

This process is continued for many epochs until the feature map does not change.

Advantages & Disadvantages

The advantages of SOM is as following :

- No backpropagation needed
- No activation function needed.
- Input nodes remains same.
- No labelling in the dataset required as it is unsupervised

The disadvantages of SOM is as following :

- Similar samples are not always near each other.
- Sometimes the clusters will get split and there will be two groups of same colored clusters.
- Lots of maps are required in order to match our clusters with the actual data.

Introduction to Deep Learning

- Deep Learning is a branch of machine learning that uses multi-layered neural networks to automatically learn complex patterns and representations from large amounts of data.
- Traditional neural networks have 1 or 2 hidden layers. Deep learning models may have **dozens or even hundreds** of layers.
- Deep learning models power most state-of-the-art artificial intelligence (AI) today, from computer vision and generative AI to self-driving cars and robotics.
- It automatically extracts useful features from raw input such as images, text or audio, which makes it highly effective for modern AI applications.
- In deep learning, the model is trained by passing input data through the network to produce an output, comparing it with the correct target using a loss function and then adjusting the weights using backpropagation and optimization techniques like gradient descent. This iterative process continues until the model learns to make accurate predictions with minimal error.

Types of deep learning models

1. Feedforward Neural Networks (ANN)

A feedforward neural network is the simplest form of deep learning where *data flows only in one direction,* from input to output through multiple hidden layers. These layers allow the model to learn increasingly complex features. They are widely used for tasks like classification and regression where structured data is involved.

2. Convolutional Neural Networks (CNN)

CNNs are specialized for *image and visual data processing*. They use convolution and pooling layers to detect local patterns such as edges, shapes, and textures. By building features layer by layer, they excel in object recognition, medical imaging, and security systems like facial recognition.

3. Recurrent Neural Networks (RNN)

RNNs *handle sequential or time-dependent data* by maintaining information from previous inputs through feedback connections. This memory-like capability makes them suitable for tasks such as speech recognition and time-series prediction. However, they struggle with long sequences due to vanishing gradients.

4. LSTM and GRU Networks

LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) networks are improved versions of RNNs designed to capture *long-term dependencies in sequences*. Using gates to control information flow, they can remember important context over long durations. They are widely used in chatbots, translation, and document processing.

5. Autoencoders

Autoencoders are unsupervised models that *learn compressed representations of data* and then reconstruct it. They are useful for noise removal, anomaly detection, and dimensionality reduction. By learning only the essential features, they help uncover hidden structure in data.

6. Generative Adversarial Networks (GANs)

GANs involve two competing networks: a generator that creates fake data and a discriminator that identifies fake from real. Through this adversarial training, GANs produce highly realistic images, videos, and synthetic datasets. They are popular in creative tasks like deepfakes and art generation.

7. Transformers

Transformers are powerful models for processing language and sequential data without relying on recurrence. They use self-attention mechanisms to understand relationships between all input elements simultaneously. Today, they are the backbone of modern NLP systems such as ChatGPT, translation tools, and summarizers.

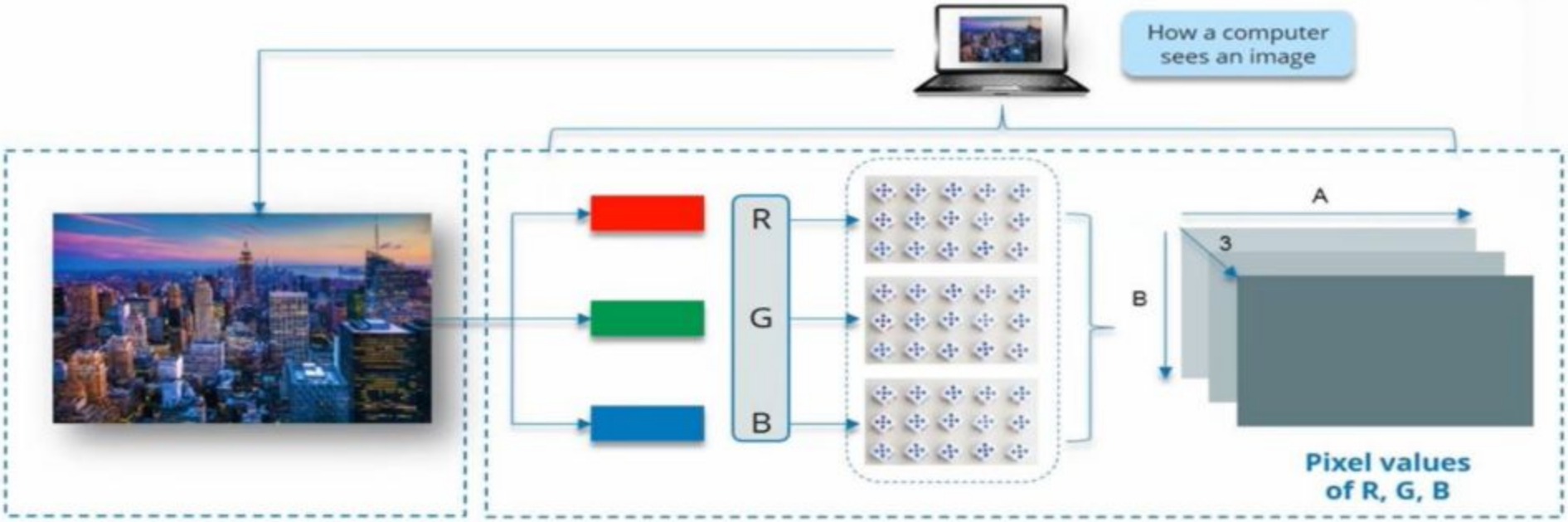
Convolutional Neural Network

In machine learning, **CNN** stands for **Convolutional Neural Network**. It is a type of deep learning algorithm specifically designed for processing and analyzing structured data like images, videos, and, in some cases, sequential data such as audio or text.

CNNs are widely used in computer vision tasks such as image classification, object detection, and segmentation.

Prior to CNNs, manual, time-consuming feature extraction methods were used to identify objects in images. However, convolutional neural networks now provide a more scalable approach to image classification and object recognition tasks, leveraging principles from linear algebra, specifically matrix multiplication, to identify patterns within an image. That said, they can be computationally demanding, requiring graphical processing units (GPUs) to train models.

How a Computer Reads an Image



Size of the image will be – B x A x 3

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

Location shifted

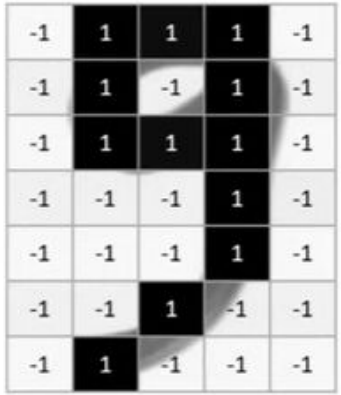
Variation 1

-1	-1	1	-1	-1
-1	1	-1	1	-1
-1	1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1

→

-1	-1	1	-1	-1
-1	1	-1	1	-1
-1	1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1
-1	-1	1	-1	-1

✗



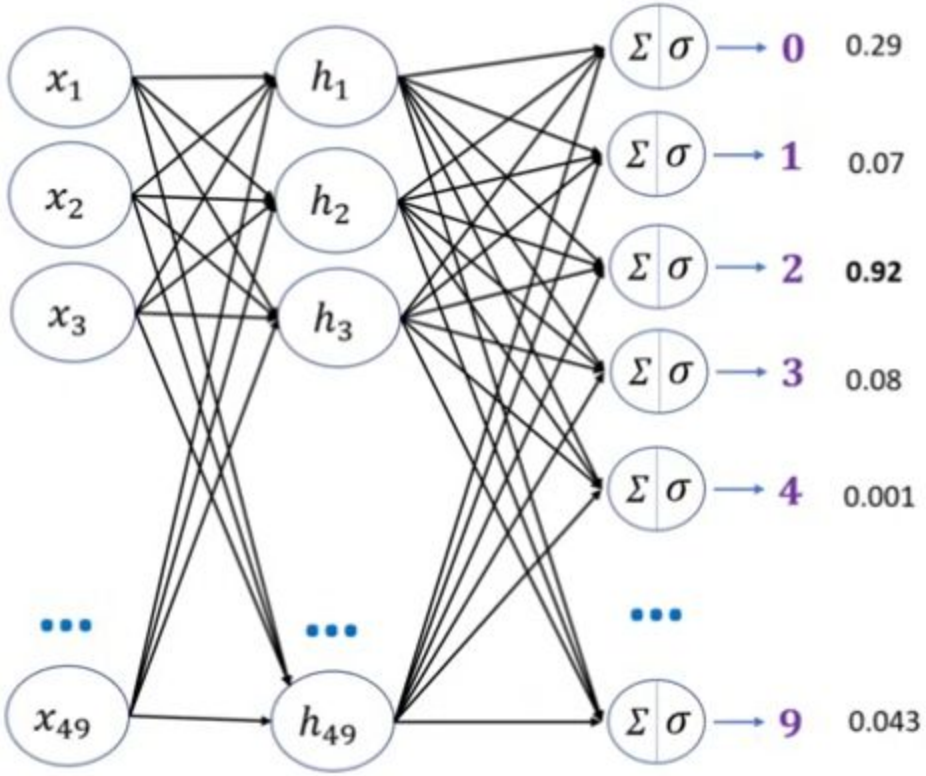
-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

To handle variety in digits we can use simple artificial neural network (ANN)

0	0	0	0	0	0	0
0	87	240	210	24	0	0
0	13	0	101	195	0	0
0	35	167	99	210	0	0
0	145	230	240	201	189	140
0	0	102	67	17	13	0
0	0	0	0	0	0	0

7 by 7 grid

0
0
0
0
0
0
0
0
87
240
210
24
0
...
0



49*49 = 2401

49*10 = 490

Image size = 1920 x 1080 X 3

First layer neurons = 1920 x 1080 X 3 ~ 6 million

Hidden layer neurons = Let's say you keep it ~ 4 million

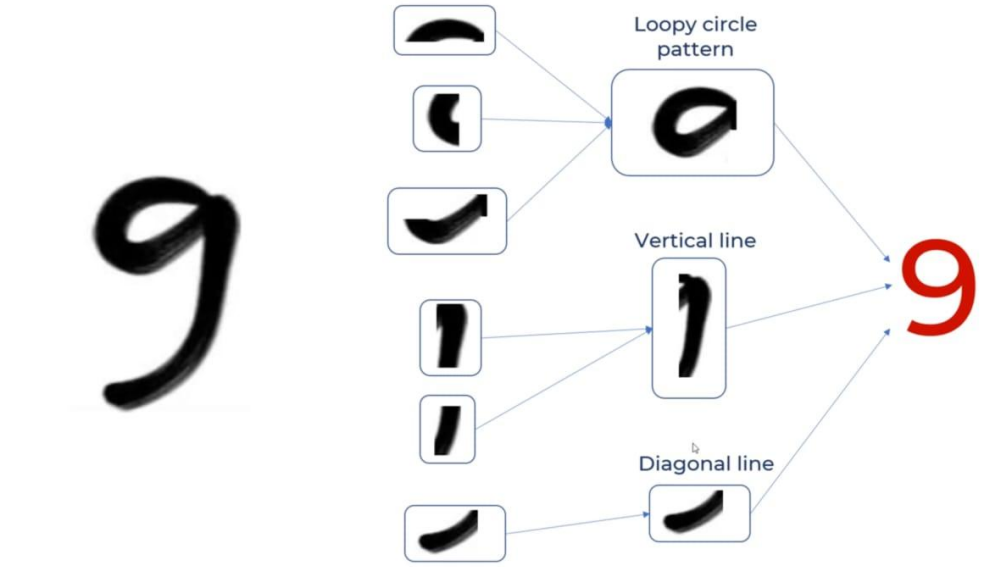
Weights between input and hidden layer = 6 mil * 4 mil
= 24 million

Disadvantage of using ANN in image classification

1. Too much computation
2. Treats local pixels same as pixels far apart
3. Sensitive to location of an object in an image

CNN

- CNN compares images piece by piece.
- The pieces that it looks for are known as features.
- By finding rough features matches, in roughly same position in two images, CNN gets a lot better at seeing the similarity than whole image matching scheme.



Types of CNN layers

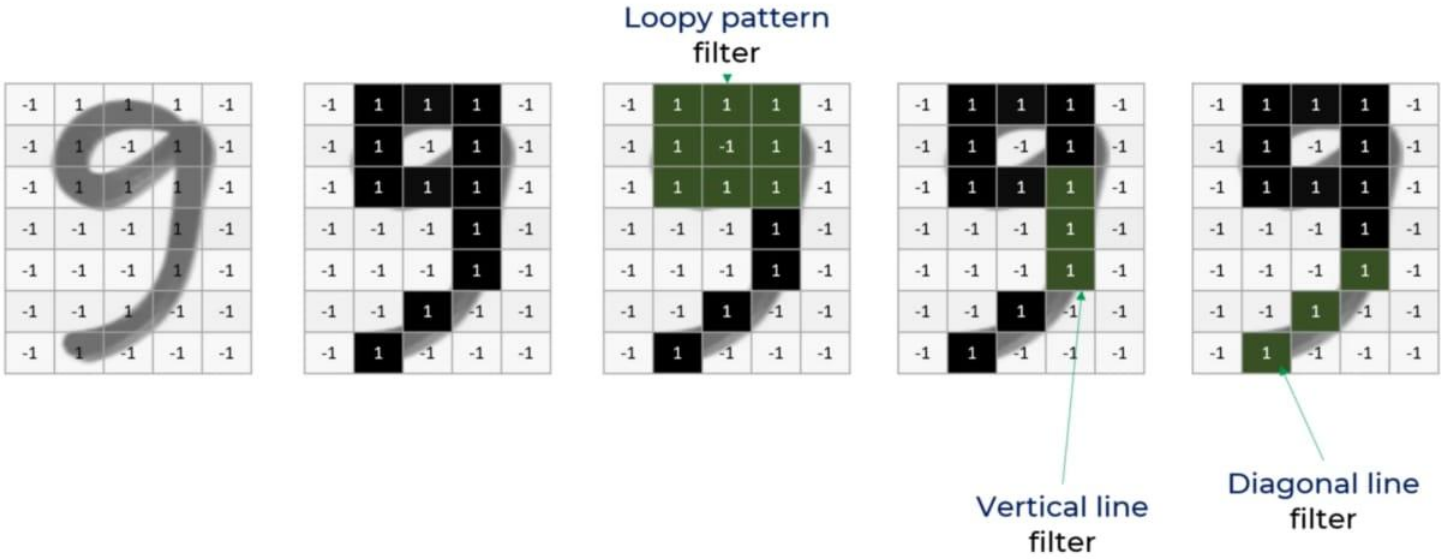
Convolutional neural networks are distinguished from other neural networks by their superior performance with image, speech or audio signal inputs. They have three main types of layers, which are:

1. Convolutional layer
2. Pooling layer
3. Fully-connected (FC) layer

- The convolutional layer is the first layer of a convolutional network.
- While convolutional layers can be followed by additional convolutional layers or pooling layers, the fully-connected layer is the final layer.
- With each layer, the CNN increases in its complexity, identifying greater portions of the image.
- Earlier layers focus on simple features, such as colors and edges. As the image data progresses through the layers of the CNN, it starts to recognize larger elements or shapes of the object until it finally identifies the intended object.

Convolutional layer

- The convolutional layer is the core building block of a CNN, and it is where the majority of computation occurs.
- These layers apply convolutional operations to extract features from the input data (e.g., edges, textures, shapes). Convolution helps preserve the spatial relationships between pixels.
- It requires a few components, which are **input data, a filter and a feature map**.
- Let's assume that the **input** will be a color image, which is made up of a **matrix of pixels in 3D**. This means that the input will have three dimensions—a height, width and depth—which correspond to RGB in an image.
- We also have a feature detector, also known as a kernel or a **filter**, which will move across the receptive fields of the image, checking if the feature is present. This process is known as a **convolution.**
- The feature detector is a two-dimensional (2-D) array of weights, which represents part of the image.
- While they can vary in size, the filter size is typically a 3x3 matrix; this also determines the size of the receptive field.


Loopy pattern filter
Vertical line filter
Diagonal line filter

- The **filter** is then applied to an area of the image, and a **dot product** is calculated between the input pixels and the filter.
- This dot product is then fed into an output array.
- Afterwards, the filter shifts by a stride, repeating the process until the kernel has swept across the entire image.
- The final output from the series of dot products from the input and the filter is known as a **feature map, activation map or a convolved feature**.

$$-1+1+1-1-1-1-1+1+1 = -1 \rightarrow -1/9 = -0.11$$

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

Input Image

Dot product

*

1	1	1
1	-1	1
1	1	1

Filter

Loopy Pattern Detector

-0.11		

Stride=1

-1	**1**	**1**	**1**	-1
-1	**1**	-1	**1**	-1
-1	**1**	**1**	**1**	-1
-1	-1	-1	**1**	-1
-1	-1	-1	**1**	-1
-1	-1	**1**	-1	-1
-1	**1**	-1	-1	-1

Input Image 5X5

*

1	1	1
1	-1	1
1	1	1

Filter 3X3

-0.11	**1**	

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

Input Image
5X5

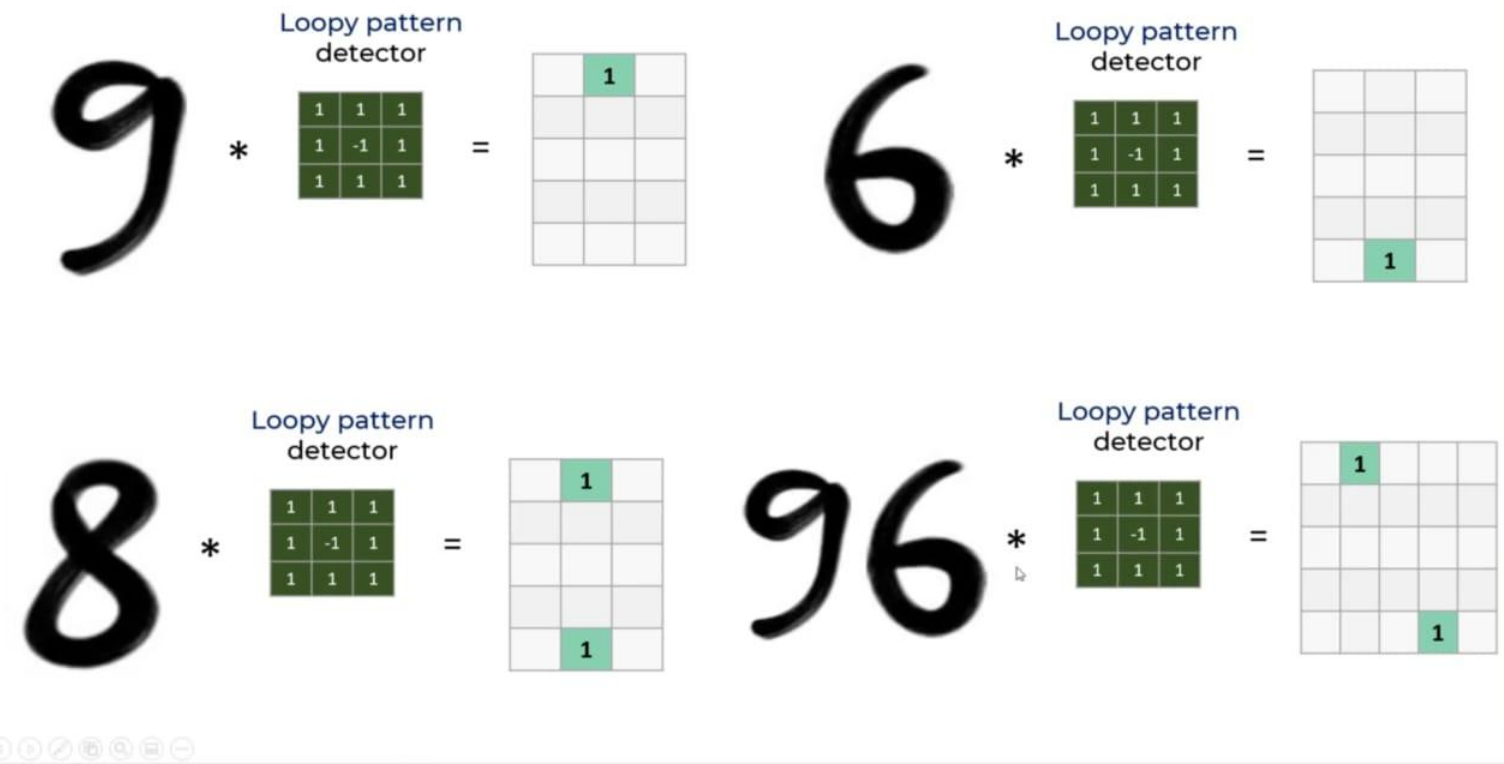
*

1	1	1
1	-1	1
1	1	1

Filter
Loopy Pattern Detector

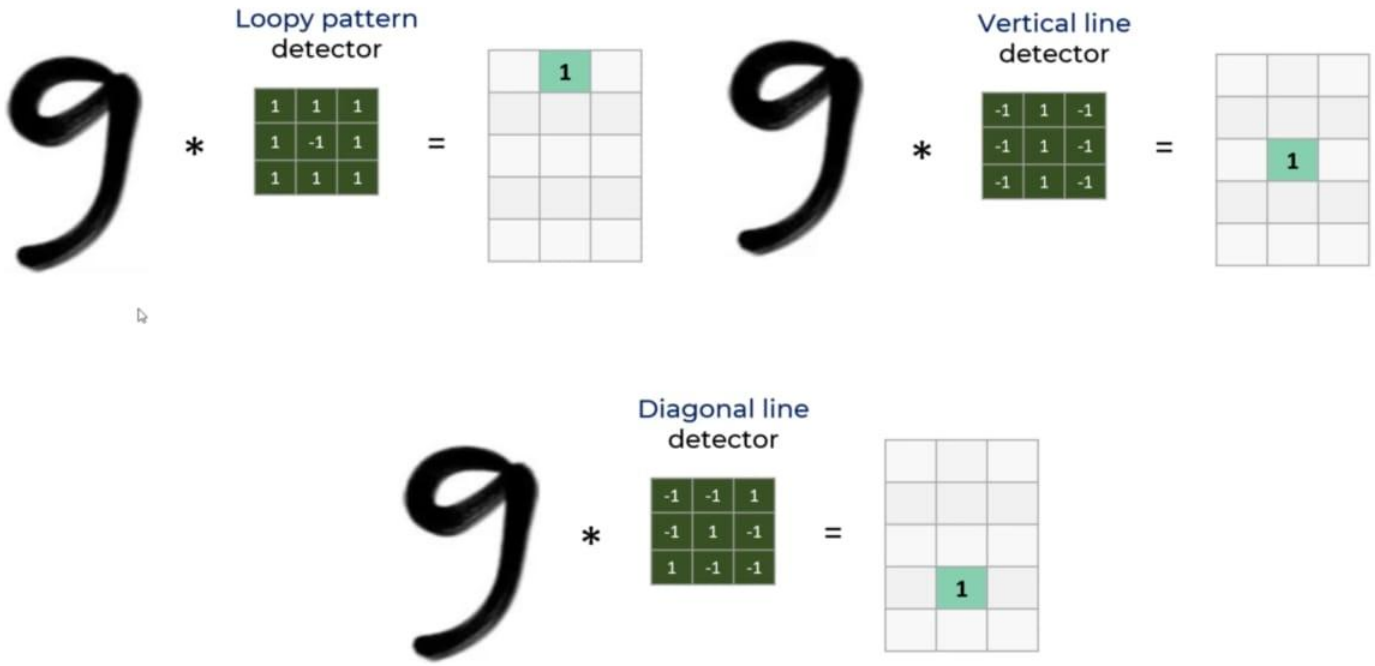
-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33

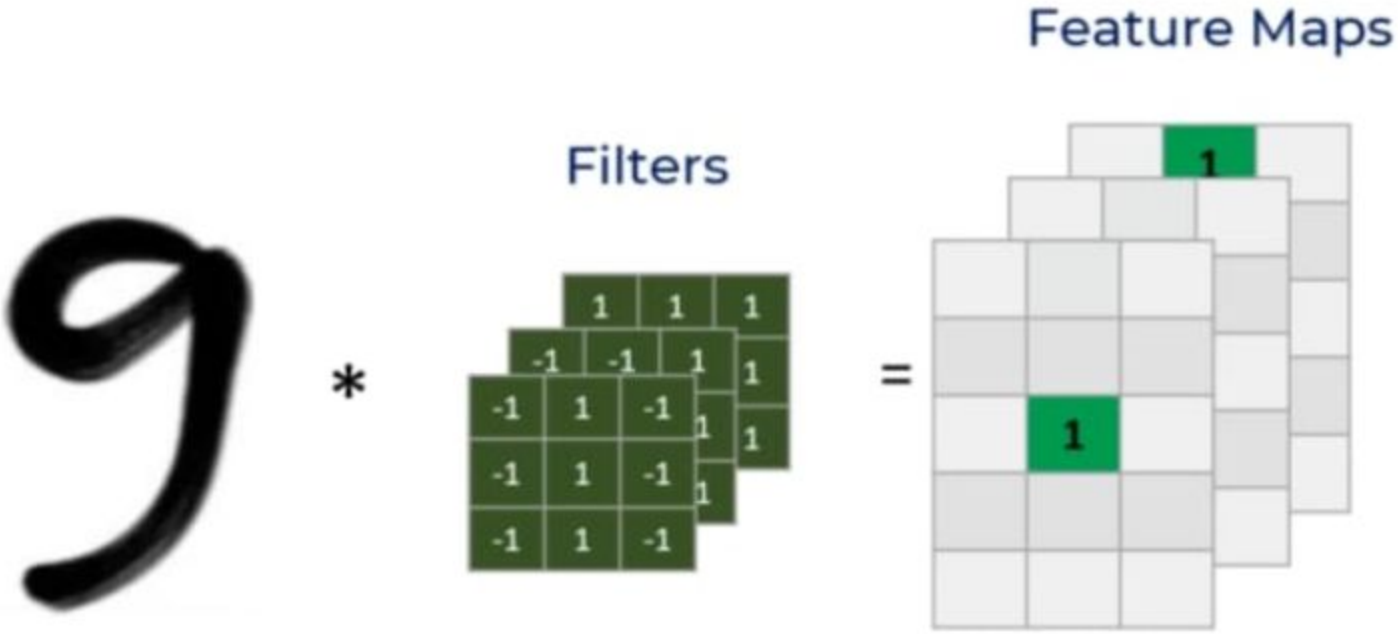
Feature Map


Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1
Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1
Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1
1
Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1
1

Additional convolutional layer

- Another convolution layer can follow the initial convolution layer. When this happens, the structure of the CNN can become hierarchical as the later layers can see the pixels within the receptive fields of prior layers.
- As an example, let's assume that we're trying to determine if an image contains a bicycle. You can think of the bicycle as a sum of parts.
- It is comprised of a frame, handlebars, wheels, pedals, and so on. Each individual part of the bicycle makes up a lower-level pattern in the neural net, and the combination of its parts represents a higher-level pattern, creating a feature hierarchy within the CNN.
- Ultimately, the convolutional layer converts the image into numerical values, allowing the neural network to interpret and extract relevant patterns.


Loopy pattern detector
1 1 1
1 -1 1
1 1 1
1
Vertical line detector
-1 1 -1
-1 1 -1
-1 1 -1
1
Diagonal line detector
-1 -1 1
-1 1 -1
1 -1 -1
1


Feature Maps
Filters
*
=
1
1 1 1
-1 -1 1
-1 1 -1
-1 1 -1
-1 1 -1

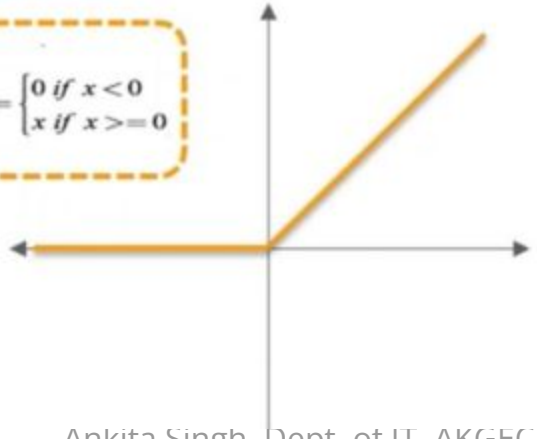
ReLU Layer

- ✓ In this layer we remove every negative values from the filtered images and replaces it with zero's
- ✓ This is done to avoid the values from summing up to zero

Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable

x	f(x)=x	F(x)
-3	f(-3) = 0	0
-5	f(-5) = 0	0
3	f(3) = 3	3
5	f(5) = 5	5

$$f(x) = \begin{cases} 0 \text{ if } x < 0 \\ x \text{ if } x >= 0 \end{cases}$$



ReLu helps to make the model non linear

-1	1	1	1	-1
-1	1	-1	1	-1
-1	1	1	1	-1
-1	-1	-1	1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	-1	-1

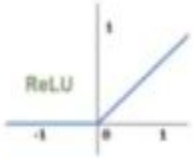
*

Loopy pattern filter

1	1	1
1	-1	1
1	1	1

→

-0.11	1	-0.11
-0.55	0.11	-0.33
-0.33	0.33	-0.33
-0.22	-0.11	-0.22
-0.33	-0.33	-0.33



→

0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

Pooling layer

Pooling layers, also known as downsampling, conducts dimensionality reduction, reducing the number of parameters in the input. Similar to the convolutional layer, the pooling operation sweeps a filter across the entire input, but the difference is that this filter does not have any weights. Instead, the kernel applies an aggregation function to the values within the receptive field, populating the output array. There are two main types of pooling:

- **Max pooling**: As the filter moves across the input, it selects the pixel with the maximum value to send to the output array. As an aside, this approach tends to be used more often compared to average pooling.
- **Average pooling**: As the filter moves across the input, it calculates the average value within the receptive field to send to the output array.

While a lot of information is lost in the pooling layer, it also has a number of benefits to the CNN. They help to reduce complexity, improve efficiency, and limit risk of overfitting.

Pooling Layer

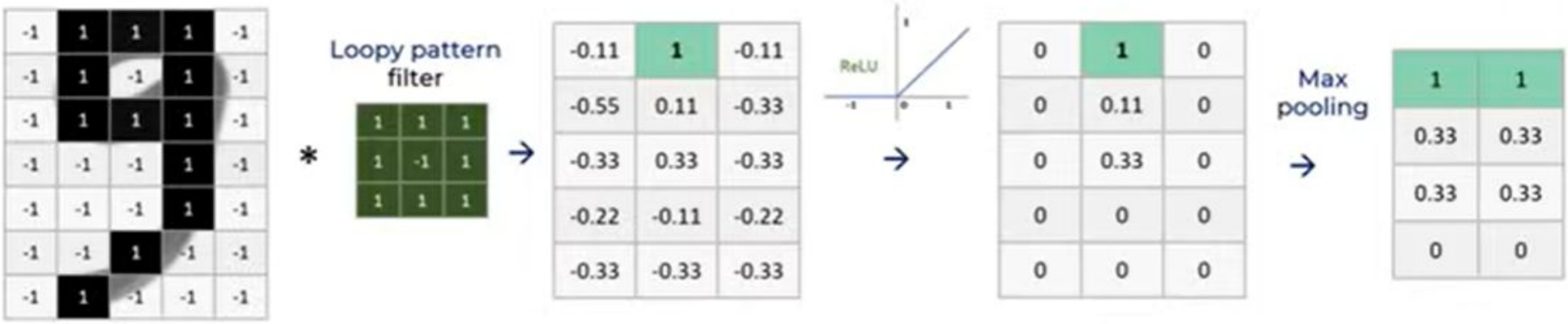
In this layer we shrink the image stack into a smaller size

Steps:

1. Pick a window size (usually 2 or 3).
2. Pick a stride (usually 2).
3. Walk your window across your filtered images.
4. From each window, take the maximum value.

Symbol:

2X2 Filter Stride=1 Max pooling



Benefits of pooling

Reduces dimensions & computation

Reduce overfitting as there are less parameters

Model is tolerant towards variations, distortions

Convolution

- Connections sparsity reduces overfitting
- Conv + Pooling gives location invariant feature detection
- Parameter sharing

ReLU

- Introduces nonlinearity
- Speeds up training, faster to compute

Pooling

- Reduces dimensions and computation
- Reduces overfitting
- Makes the model tolerant towards small distortion and variations

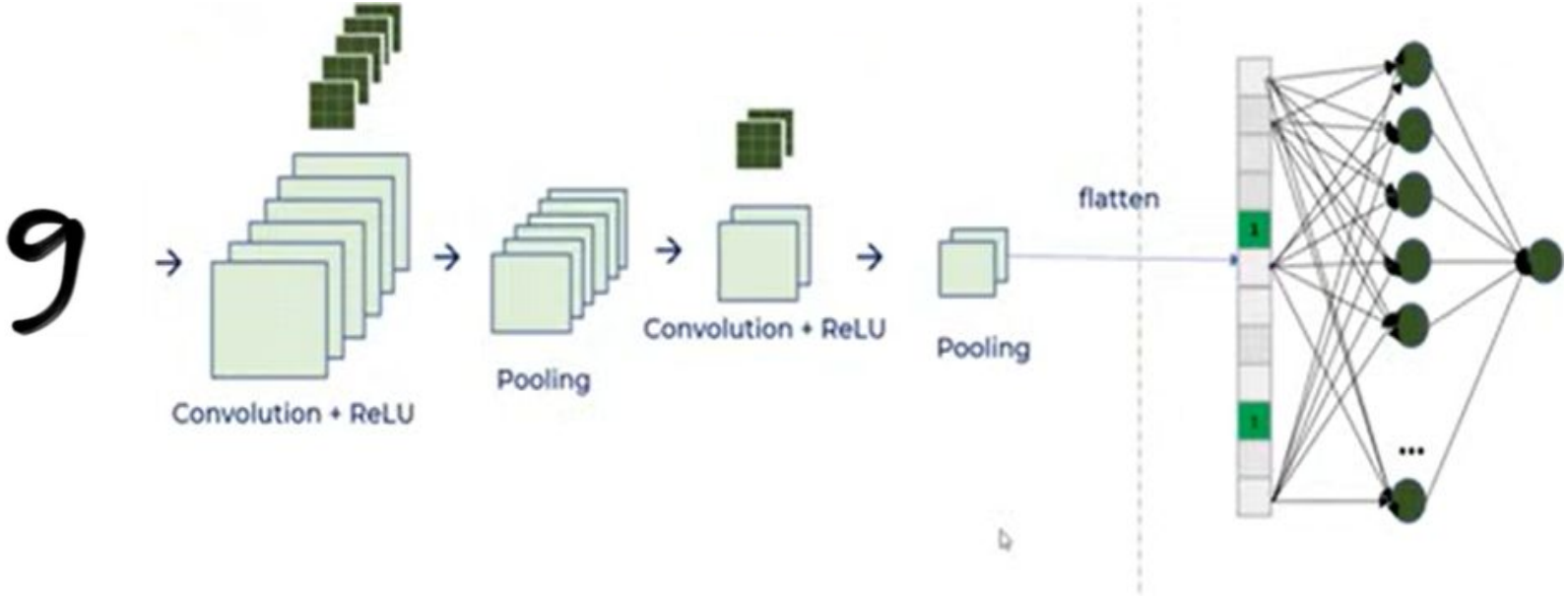
Fully-connected layer

- As mentioned, the pixel values of the input image are not directly connected to the output layer in partially connected layers. However, in the fully-connected layer, each node in the output layer connects directly to a node in the previous layer.

- This layer performs the task of classification based on the features extracted through the previous layers and their different filters.
- While convolutional and pooling layers tend to use ReLu functions, FC layers usually leverage a softmax activation function to classify inputs appropriately, producing a probability from 0 to 1.

Dropout:

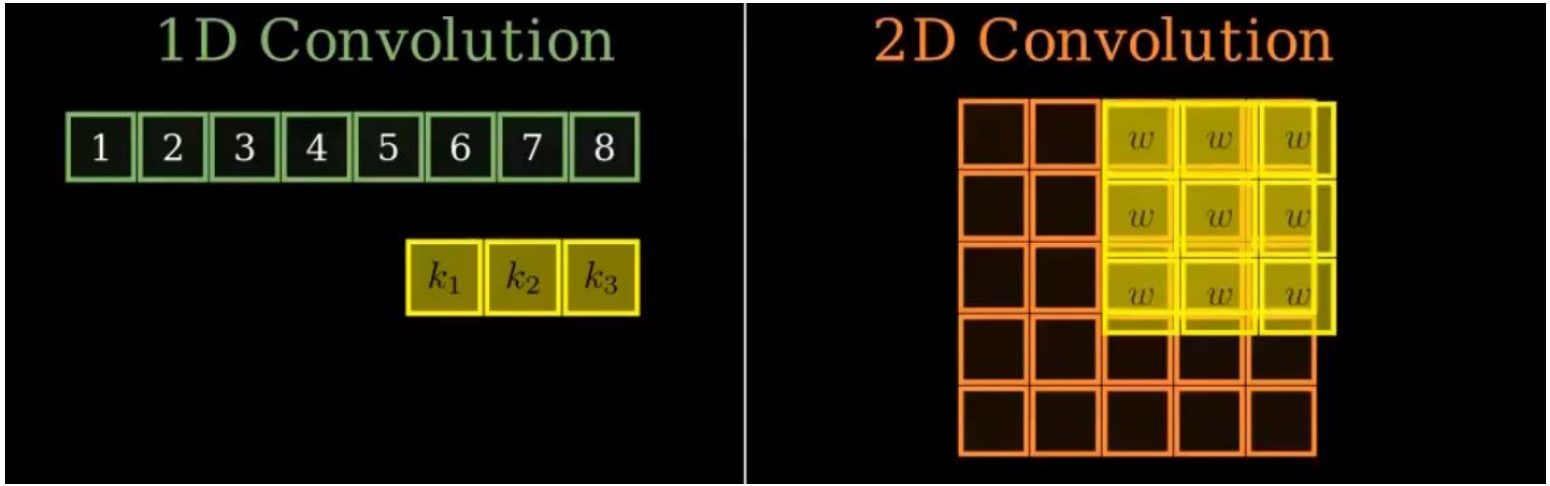
  - It is a regularization technique used to prevent overfitting by randomly deactivating neurons during training.



Is number 9 or not?

Concept of Convolution (1D and 2D) layers

- 1D convolutions process sequential data like time series or text by sliding a kernel along a single axis, while 2D convolutions handle spatial data like images by sliding a kernel across two axes (height and width).
- This means 1D layers capture temporal patterns and output a 1D array, whereas 2D layers extract spatial features and produce a 2D feature map

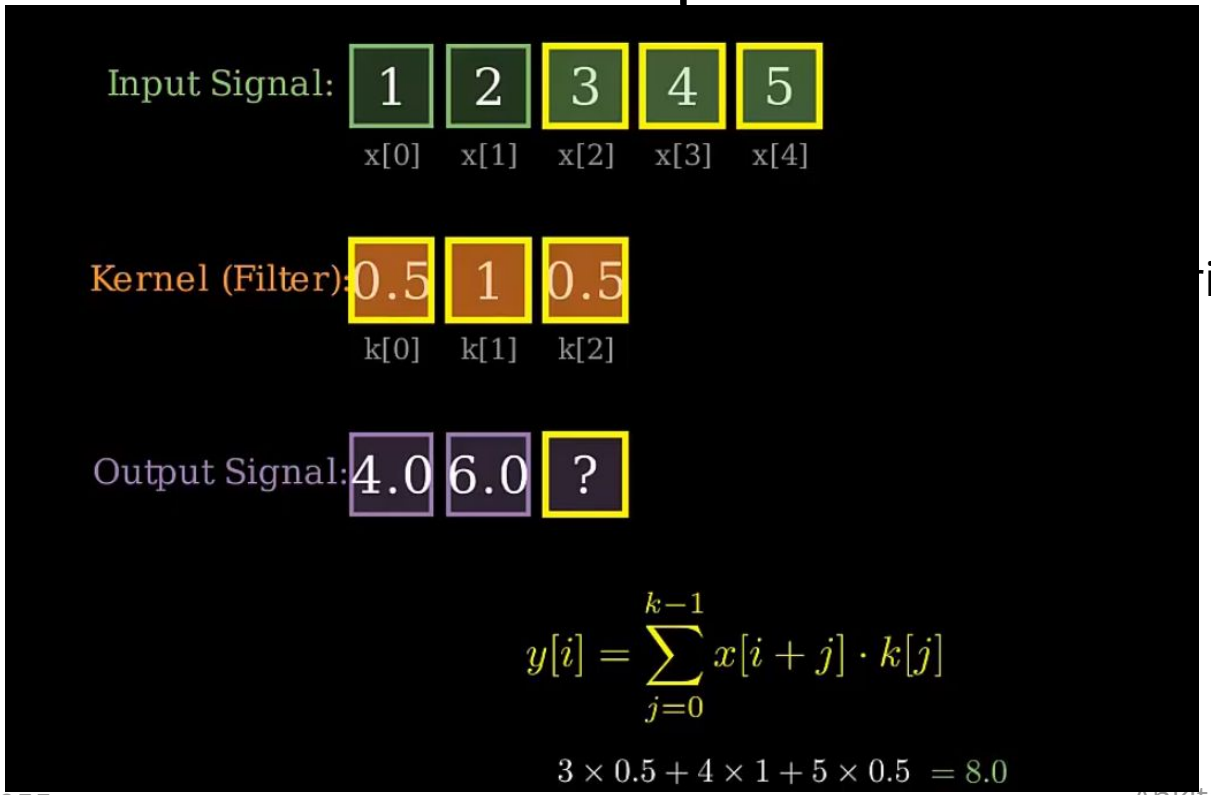


Ankita Singh, Dept. of IT, AKGEC

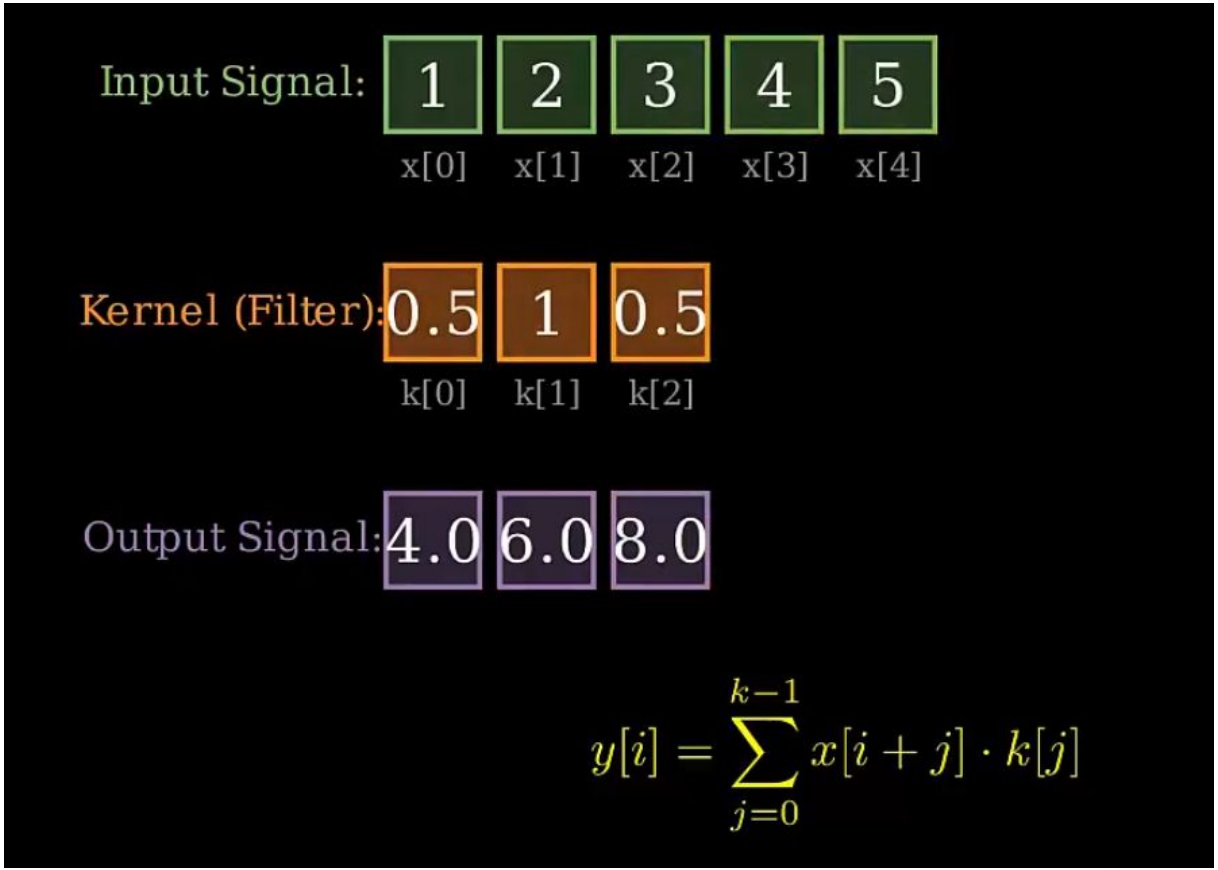
1D Convolutional Layers

- Input data: Sequential data, such as time series, audio signals, or text.
- Kernel: A 1D filter or kernel that slides along a single axis of the input.
- Operation: Applies the kernel across the sequence to detect temporal features.
- Output: A 1D feature map.
- Use case: Extracting patterns from sequences, like identifying a specific sound in an audio clip or recognizing a word in a sentence.

1D Convolution- Example

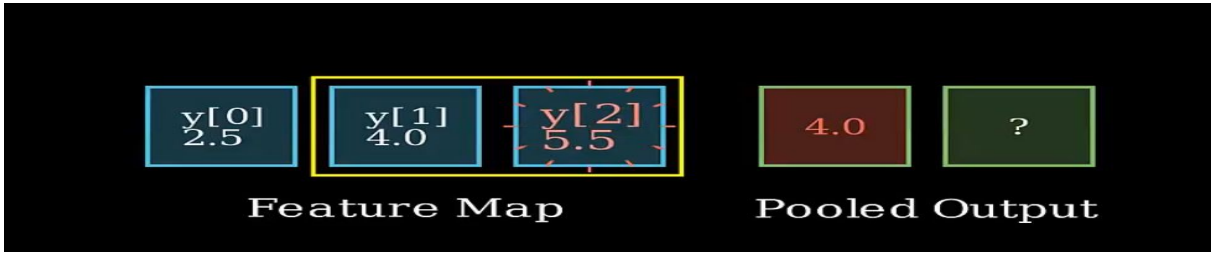


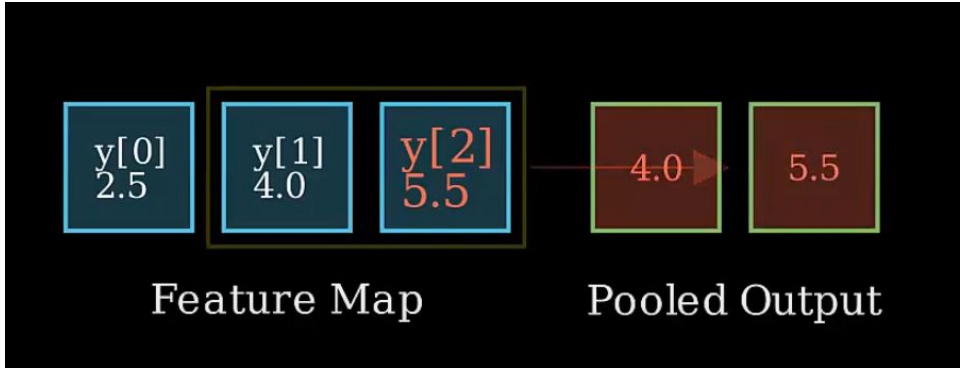
ride=1

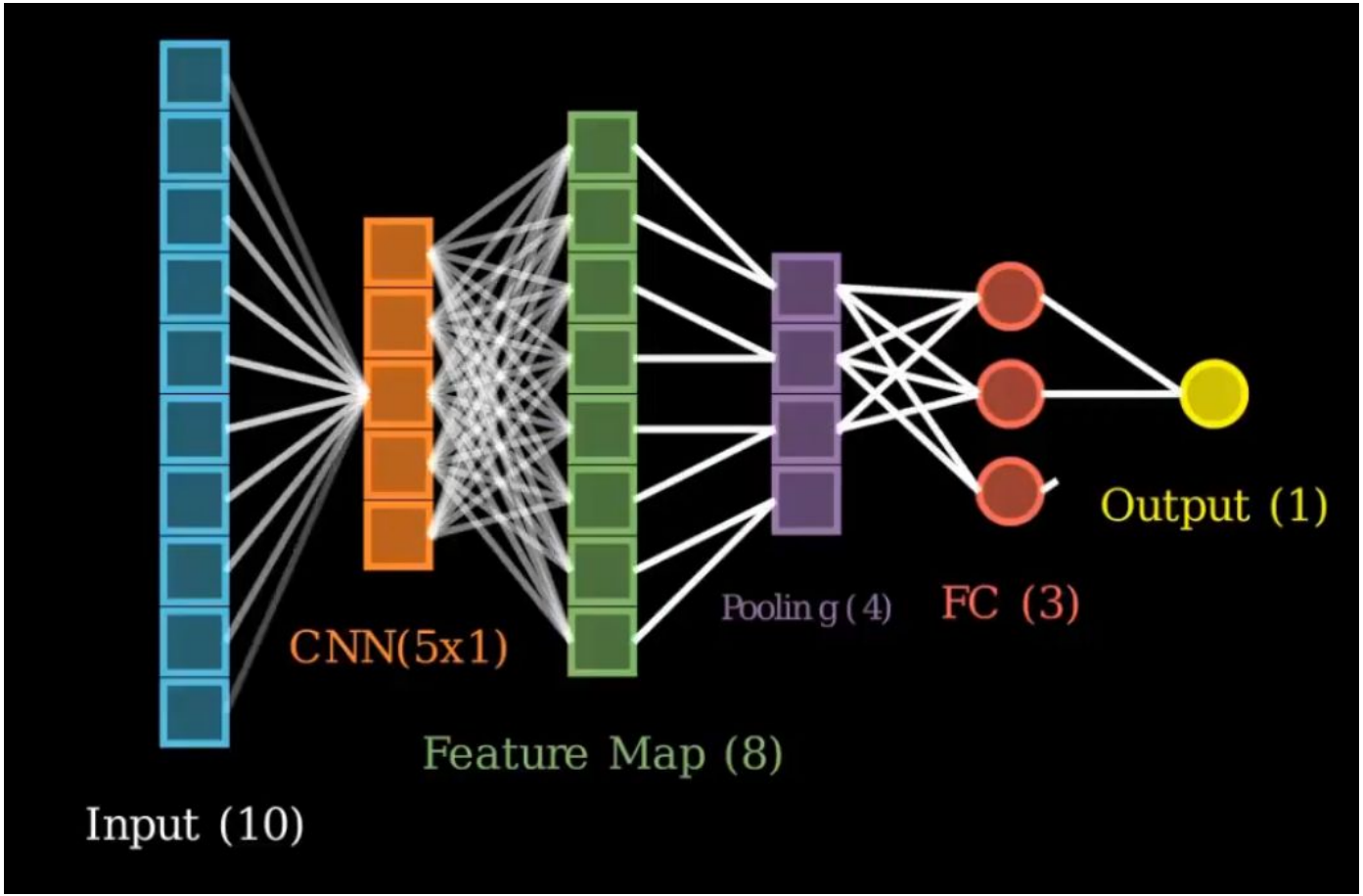

Input Signal:
1 2 3 4 5
x[0] x[1] x[2] x[3] x[4]
Kernel (Filter):
0.5 1 0.5
k[0] k[1] k[2]
Output Signal:
4.0 6.0 8.0
$y[i] = \sum_{j=0}^{k-1} x[i+j] \cdot k[j]$

Stride=1

Max Pooling


y[0]
2.5
y[1]
4.0
y[2]
5.5
4.0
?
Feature Map
Pooled Output


y[0]
2.5
y[1]
4.0
y[2]
5.5
4.0
5.5
Feature Map
Pooled Output


Input (10)
CNN(5x1)
Feature Map (8)
Pooling (4)
FC (3)
Output (1)

2D Convolutional Layers

- Input data: 2D data, primarily images.
- Kernel: A 2D filter that slides across two axes: height and width.
- Operation: Applies the kernel across the grid to identify spatial patterns and features.
- Output: A 2D feature map that preserves the spatial relationships of the original data.
- Use case: Classifying objects in images by detecting edges, textures, and shapes.

How to Evaluate CNN Models

Efficiency of CNN can be evaluated using a variety of criteria. Among the most popular metrics are:

- **Accuracy:** Accuracy is the percentage of test images that the CNN correctly classifies.
- **Precision:** Precision is the percentage of test images that the CNN predicts as a particular class and that are actually of that class.
- **Recall:** Recall is the percentage of test images that are of a particular class and that the CNN predicts as that class.
- **F1 Score:** The F1 Score is a harmonic mean of precision and recall. It is a good metric for evaluating the performance of a CNN on classes that are imbalanced.

Different Types of CNN Models

1. LeNet
2. AlexNet
3. Resnet
4. GoogleNet
5. VGG

Applications

- **Image classification:** CNNs are the state-of-the-art models for image classification. They can be used to classify images into different categories such as cats and dogs.
- **Object detection:** It can be used to detect objects in images such as people, cars and buildings. They can also be used to localize objects in images which means that they can identify the location of an object in an image.
- **Image segmentation:** It can be used to segment images which means that they can identify and label different objects in an image. This is useful for applications such as medical imaging and robotics.
- **Video analysis:** It can be used to analyze videos such as tracking objects in a video or detecting events in a video. This is useful for applications such as video surveillance and traffic monitoring.

Advantages:

- Automatically learns relevant features without manual feature engineering.
- Efficient in processing grid-like data structures.

Limitations:

- Requires large datasets and significant computational resources.
- Sensitive to hyperparameter tuning and architecture design.

CNNs have revolutionized the field of computer vision, making them a cornerstone of modern AI systems.